E-Commerce Platform Search Function

Big O Notation

Big O Notation is a mathematical representation used to analyze the efficiency of algorithms. It describes how the execution time or memory usage of an algorithm grows as the size of the input increases. This helps developers compare different algorithms and select the most efficient solution for a given problem.

Common time complexities include:

- **O(1) – Constant Time:** Execution time remains the same regardless of input size. Example: Accessing an array element by its index.
- **O(log n) – Logarithmic Time:** Execution time increases slowly as the input size grows. Example: Binary Search.
- **O(n) – Linear Time:** Execution time grows proportionally with the input size. Example: Linear Search.
- **O(n log n) – Linearithmic Time:** Common in efficient sorting algorithms such as Merge Sort and Heap Sort.
- **$O(n^2)$ – Quadratic Time:** Execution time increases significantly with larger inputs. Example: Bubble Sort and Insertion Sort.

Time Complexity Analysis

1. Linear Search

Linear Search traverses the array sequentially until the required product ID is found or the end of the array is reached.

Complexity Analysis:

- **Best Case:** O(1) – The target product is located at the first position.
- **Average Case:** O(n) – The target product is typically found after examining approximately half of the elements.
- **Worst Case:** O(n) – The target product is either the last element or does not exist in the array.

2. Binary Search

Binary Search repeatedly divides the sorted array into two halves and compares the target product ID with the middle element to determine the next search range.

Complexity Analysis:

- **Best Case:** O(1) – The target product is the middle element.
- **Average Case:** O(log n) – The search space is reduced by half during each iteration.
- **Worst Case:** O(log n) – The maximum number of divisions is required before finding the product or determining that it is not present.

Conclusion

Both Linear Search and Binary Search can be used to locate products within an e-commerce platform. However, Binary Search is significantly more efficient for large datasets because it reduces the search space by half during each iteration. While Linear Search has a time complexity of O(n), Binary Search operates in O(log n) time, making it the preferred choice when the product data is sorted. Therefore, Binary Search provides faster search performance and better scalability for large e-commerce applications.